R.Vishnu Vardhan - 2520090166

SKILL_WEEK-2

1) Task 1:

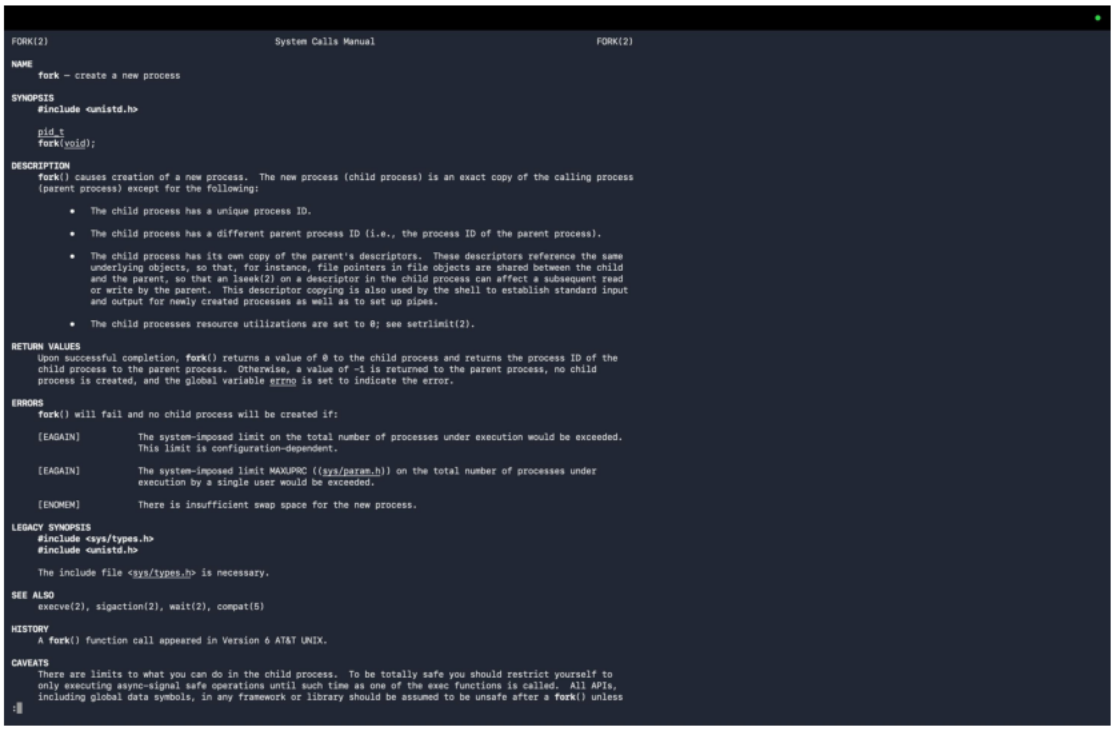
```
Last login: Wed Aug 12 20:53:13 on ttys007
root@Vishnu:~#                  mkdir OSSP
mkdir: OSSP: File exists
root@Vishnu:~#                  cd OSSP
root@Vishnu:~#                     pwd
/Users/amrutha/OSSP
root@Vishnu:~#                     mkdir src include obj bin docs tests scripts assets
mkdir: src: File exists
mkdir: include: File exists
mkdir: obj: File exists
mkdir: bin: File exists
mkdir: docs: File exists
mkdir: tests: File exists
mkdir: scripts: File exists
mkdir: assets: File exists
```

Figure 1: OSSP project directory created in mac-OS Terminal

```
root@Vishnu:~#                          find . -maxdepth 2 -not -path './.git*' -print
.
./obj
./hello.c
./LICENSE
./bin
./bin/my_shell
./Makefile
./include
./include/shell.h
./include/builtin.h
./include/parser.h
./include/executor.h
./tests
./tests/test_executor.c
./tests/test_parser.c
./docs
./docs/design.md
./docs/report.pdf
./README.md
./scripts
./scripts/run.sh
./hello
./assets
./assets/architecture.png
./src
./src/executor.c
./src/main.c.save
./src/shell.c
./src/main.c
./src/builtin.c
./src/parser.c
```

Figure 2 & 3: OSSP project structure with source, header, documentation, test, script and asset directories

2)Task 1:



```
FORK(2)                          System Calls Manual                          FORK(2)

NAME
     fork — create a new process

SYNOPSIS
     #include <unistd.h>

     pid_t
     fork(void);

DESCRIPTION
     fork() causes creation of a new process.  The new process (child process) is an exact copy of the calling process
     (parent process) except for the following:

           •  The child process has a unique process ID.

           •  The child process has a different parent process ID (i.e., the process ID of the parent process).

           •  The child process has its own copy of the parent's descriptors.  These descriptors reference the same
              underlying objects, so that, for instance, file pointers in file objects are shared between the child
              and the parent, so that an lseek(2) on a descriptor in the child process can affect a subsequent read
              or write by the parent.  This descriptor copying is also used by the shell to establish standard input
              and output for newly created processes as well as to set up pipes.

           •  The child processes resource utilizations are set to 0; see setrlimit(2).

RETURN VALUES
     Upon successful completion, fork() returns a value of 0 to the child process and returns the process ID of the
     child process to the parent process.  Otherwise, a value of -1 is returned to the parent process, no child
     process is created, and the global variable errno is set to indicate the error.

ERRORS
     fork() will fail and no child process will be created if:

     [EAGAIN]           The system-imposed limit on the total number of processes under execution would be exceeded.
                        This limit is configuration-dependent.

     [EAGAIN]           The system-imposed limit MAXUPRC ((sys/param.h)) on the total number of processes under
                        execution by a single user would be exceeded.

     [ENOMEM]           There is insufficient swap space for the new process.

LEGACY SYNOPSIS
     #include <sys/types.h>
     #include <unistd.h>

     The include file <sys/types.h> is necessary.

SEE ALSO
     execve(2), sigaction(2), wait(2), compat(5)

HISTORY
     A fork() function call appeared in Version 6 AT&T UNIX.

CAVEATS
     There are limits to what you can do in the child process.  To be totally safe you should restrict yourself to
     only executing async-signal safe operations until such time as one of the exec functions is called.  All APIs,
     including global data symbols, in any framework or library should be assumed to be unsafe after a fork() unless
:
```

```
EXEC(3)                      Library Functions Manual                      EXEC(3)

NAME
     execl, execle, execlp, execv, execvp, execvP — execute a file

LIBRARY
     Standard C Library (libc, -lc)

SYNOPSIS
     #include <unistd.h>

     extern char **environ;

     int
     execl(const char *path, const char *arg0, ..., /*, (char *)0, */);

     int
     execle(const char *path, const char *arg0, ..., /* (char *)0 char *const envp[] */);

     int
     execlp(const char *file, const char *arg0, ..., /*, (char *)0, */);

     int
     execv(const char *path, char *const argv[]);

     int
     execvp(const char *file, char *const argv[]);

     int
     execvP(const char *file, const char *search_path, char *const argv[]);

DESCRIPTION
     The exec family of functions replaces the current process image with a new process image.  The functions described in this manual page are front-ends for the function execve(2).  (See the manual page
     for execve(2) for detailed information about the replacement of the current process.)

     The initial argument for these functions is the pathname of a file which is to be executed.

     The const char *arg0 and subsequent ellipses in the execl(), execlp(), and execle() functions can be thought of as arg0, arg1, ..., argn.  Together they describe a list of one or more pointers to
     null-terminated strings that represent the argument list available to the executed program.  The first argument, by convention, should point to the file name associated with the file being executed.
     The list of arguments must be terminated by a NULL pointer.

     The execv(), execvp(), and execvP() functions provide an array of pointers to null-terminated strings that represent the argument list available to the new program.  The first argument, by convention,
     should point to the file name associated with the file being executed.  The array of pointers must be terminated by a NULL pointer.

     The execle() function also specifies the environment of the executed process by following the NULL pointer that terminates the list of arguments in the argument list or the pointer to the argv array
     with an additional argument.  This additional argument is an array of pointers to null-terminated strings and must be terminated by a NULL pointer.  The other functions take the environment for the
     new process image from the external variable environ in the current process.

     Some of these functions have special semantics.

     The functions execlp(), execvp(), and execvP() will duplicate the actions of the shell in searching for an executable file if the specified file name does not contain a slash */* character.  For
     execlp() and execvp(), search path is the path specified in the environment by "PATH" variable.  If this variable is not specified, the default path is set according to the _PATH_DEFPATH definition in
     <paths.h>, which is set to "/usr/bin:/bin".  For execvP(), the search path is specified as an argument to the function.  In addition, certain errors are treated specially.

     If an error is ambiguous (for simplicity, we shall consider all errors except ENOEXEC as being ambiguous here, although only the critical error EACCES is really ambiguous), then these functions will
     act as if they stat the file to determine whether the file exists and has suitable execute permissions.  If it does, they will return immediately with the global variable errno restored to the value
     set by execve().  Otherwise, the search will be continued.  If the search completes without performing a successful execve() or terminating due to an error, these functions will return with the global
     variable errno set to EACCES or ENOENT according to whether at least one file with suitable execute permissions was found.

:
```

```
amrutha@Amruthas-root@Vishnu:~# -k fork
ResMerger(1), /ut...1) - Merges resource forks or files into one resource file (DEPRECATED)
SplitForks(1), /usr/bin/SplitForks(1) - Divide a two-fork HFS file into AppleDouble format resource and data files. (DEPRECATED)
pg_rewind(1)            - synchronize a PostgreSQL data directory with another data directory that was forked from it
OPENSSL_fork_prepare(3ssl), OPENSSL_fork_parent(3ssl), OPENSSL_fork_child(3ssl) - OpenSSL fork handlers
pg_rewind(1)            - synchronize a PostgreSQL data directory with another data directory that was forked from it
perlfork(1)             - Perl's fork() emulation
spfd(1)                 - simple forking daemon to provide SPF query services
ModPerl::PerlRunPrefork(3pm) - Run unaltered CGI scripts under mod_perl
ModPerl::RegistryPrefork(3pm) - Run unaltered CGI scripts under mod_perl
Net::Server::Daemonize(3pm) - Safe fork and daemonization utilities
Net::Server::Fork(3pm)   - Net::Server personality
Net::Server::PreFork(3pm) - Net::Server personality
Net::Server::PreForkSimple(3pm) - Net::Server personality
Test2::IPC(3pm)         - Turn on IPC for threading or forking support
Test2::Require::Fork(3pm) - Skip a test file unless the system supports forking
Test2::Require::RealFork(3pm) - Skip a test file unless the system supports true forking
fork(2)                 - create a new process
openpty(3), login_tty(3), forkpty(3) - tty utility functions
perlfork(1)             - Perl's fork() emulation
pthread_atfork(3)       - register handlers to be called before and after fork
spfd(1)                 - simple forking daemon to provide SPF query services
vfork(2)                - deprecated system call to create a new process
```

Figure: Manual search for fork() system call using man -k fork

Task 2:

```
  UW PICO 5.09                                File: src/main.c

#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>
#include <string.h>

int main()
{
    pid_t pid;
    char command[100];

    printf("Enter a command: ");
    fgets(command, sizeof(command), stdin);

    command[strcspn(command, "\n")] = '\0';

    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed!\n");
        return 1;
    }
    else if (pid == 0)
    {
        printf("\nChild Process\n");
        printf("Child PID  : %d\n", getpid());
        printf("Parent PID : %d\n", getppid());

        printf("\nExecuting command: %s\n", command);

        execlp(command, command, (char *)NULL);

        printf("Exec failed!\n");
        return 1;
    }
    else
    {
        printf("\nParent Process\n");
        printf("Parent PID : %d\n", getpid());
        printf("Child PID  : %d\n", pid);

        wait(NULL);

        printf("\nChild process completed.\n");
    }

    return 0;
}

^G Get Help    ^O WriteOut    ^R Read File   ^Y Prev Pg    ^K Cut Text    ^C Cur Pos
^X Exit        ^J Justify     ^W Where is    ^V Next Pg    ^U UnCut Text  ^T To Spell
```

Figure 1: C program implementing fork() and exec()

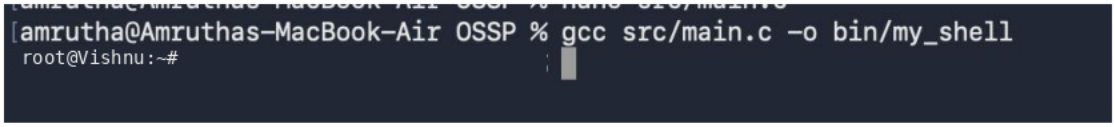


Figure 2: Successful compilation of the C program

Figure 3: Execution of ls command

Figure 4: Execution of pwd command

fork()
pid = fork();
Creates a new child process.

exec()
execlp(command, command, (char *)NULL);
Replaces the child process with the program corresponding to the command entered by the user.

wait()
wait(NULL);
Makes the parent process wait for the child process to finish.

getpid()
getpid()
Returns the PID of the current process.

getppid()
getppid()
Returns the PID of the parent process.

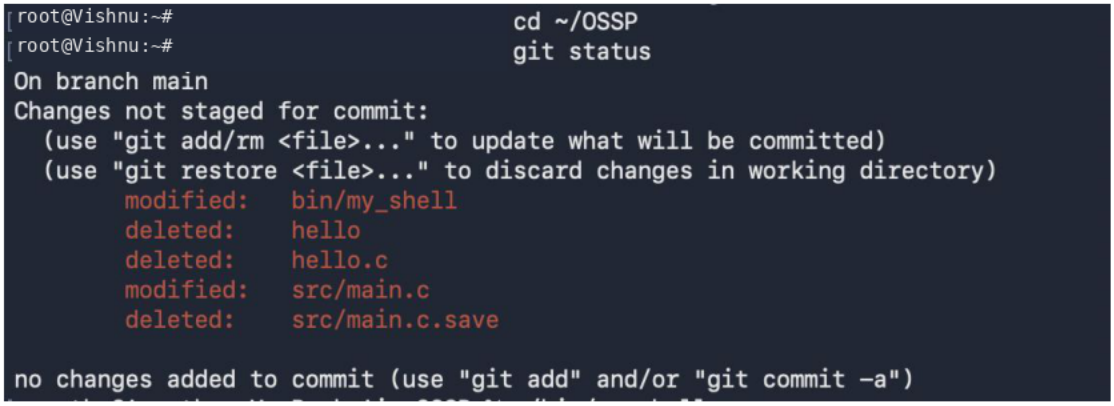


Figure 5: Git status showing changes made for Task 2 and removal of extra files.